

Forjai — Arquitectura de la aplicación

Sep 30, 2026 · @alexander pineda

Resumen

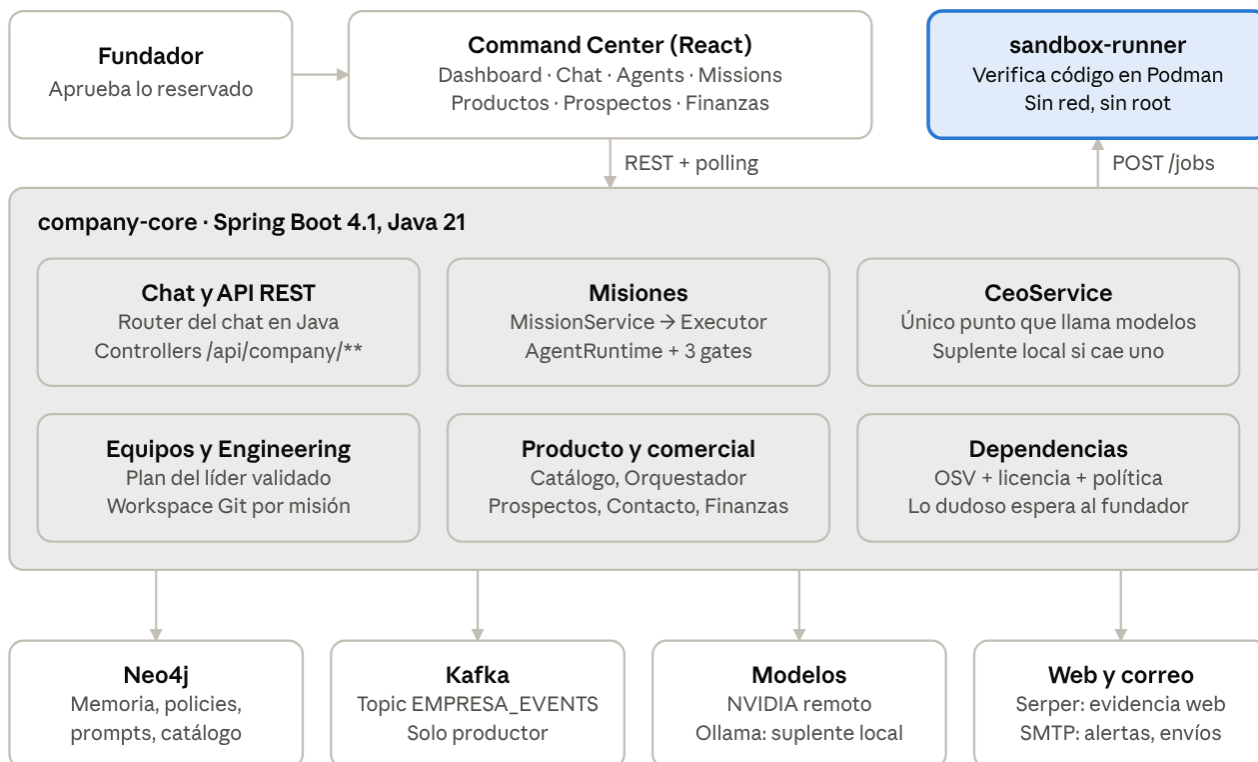
Forjai es una empresa real operada por 14 agentes de IA: investigan mercados, diseñan y construyen productos y buscan clientes, mientras el fundador aprueba lo reservado. Técnicamente es un monolito Spring Boot 4.1 (Java 21), `company-core`, con una SPA React como Command Center, memoria en Neo4j, eventos en Kafka y modelos en Ollama o NVIDIA. El código que generan los agentes nunca corre en `company-core`: lo verifica un servicio aparte, `sandbox-runner`, en contenedores Podman sin red.

El hilo conductor es **determinismo sobre LLM**: todo lo que Java puede decidir (ruteo del chat, cálculos, validación, contradicciones, finanzas) lo decide Java, nunca el modelo revisándose a sí mismo.

Quién	Puede hacer	Ejemplos
Agentes (●)	Investigar, proponer, construir, buscar prospectos, redactar correos	Discovery, misiones de equipo, orquestador, búsqueda diaria
Fundador (●)	Cerrar misiones, aprobar envíos y dependencias, registrar ventas y gastos, pausar o retirar productos	<code>POST /missions/{id}/decision</code> , correos por aprobar, Finanzas

Arquitectura

company-core orquesta todo; solo sandbox-runner ejecuta el código generado

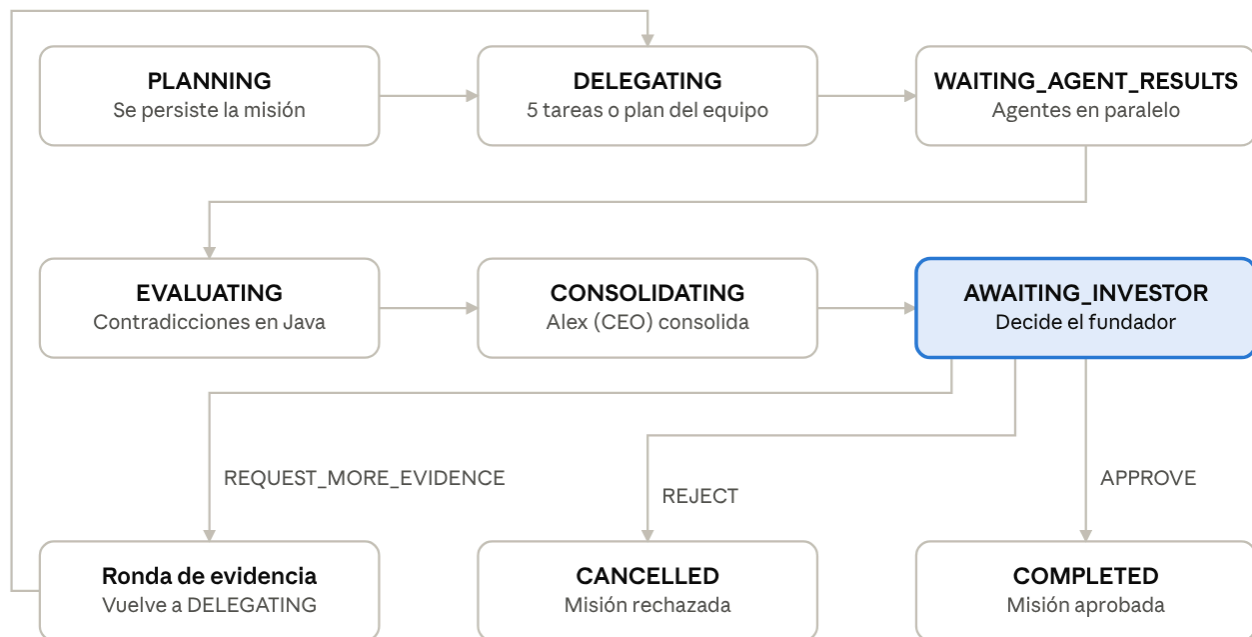


arquitectura · 2 servicios, 4 dependencias externas

El fundador opera todo desde el Command Center, que habla por REST con `company-core`; este persiste en Neo4j, publica en Kafka, llama a los modelos y delega la verificación del código al `sandbox-runner`, el único que toca Podman.

Flujo de una misión

Una misión nunca se cierra sola: la cierra el fundador



estados de una misión · 6 etapas, 3 decisiones del fundador

`MissionExecutor.advanceMission` es el único punto que persiste cada transición, publica a Kafka y avisa por correo. Cualquier excepción lleva a `FAILED`, igual que si fallan todos los agentes; si falla solo uno, se le da 1 re-ejecución y la misión sigue con los demás. Desde `FAILED` también se puede pedir una ronda de evidencia, hasta `MAX_EVIDENCE_ROUNDS`.

Cada resultado de agente pasa tres gates en Java antes de aceptarse (hasta 3 intentos, con el motivo del rechazo en el siguiente prompt):

Gate	Revisa	Si falla
<code>AgentResultValidator</code>	Campos obligatorios, confianza, recalcula cada cálculo	Reintenta
<code>EvidenceValidationGate</code>	Tipo de fuente, evidencia verificada con URL real	Falla la tarea
<code>EvidenceBindingGate</code>	Si buscó en la web, que cite al menos una URL	Reintenta

Misiones por equipo y Engineering

Una misión con `teamId` no crea las 5 tareas fijas: el líder del equipo planifica y Java valida el plan (`TeamPlanValidator` , 3 intentos, sin plan por defecto). La estrategia depende del tipo de equipo.

Equipo	Estrategia	Qué produce
Creative / Product Intelligence	<code>AnalysisTeamStrategy</code> (agentes en paralelo)	Análisis, ofertas, entrega de servicios
Marketing & Growth	<code>AnalysisTeamStrategy</code>	Estrategias, contenido
Engineering	<code>DevelopmentTeamStrategy</code> (secuencial por capas)	Repositorio Git verificado en sandbox

Engineering trabaja en orden fijo de capas DDD, cada agente viendo lo ya commiteado:

1. Forjai commitea el scaffold del perfil (`DOTNET_APP` , `GODOT_DOTNET_GAME` o `FLUTTER_WEB_APP`).
2. Neo genera domain; Iris application e infrastructure; Mila API, juego o presentación; Diego los tests. Un commit por agente.
3. `StaticWorkspaceValidator` revisa Git, estructura del perfil y dependencias entre capas (`DddLayerChecker`).
4. Las dependencias nuevas pasan por `DependencyService` : OSV, licencia y política; lo no aprobable espera al fundador.
5. `sandbox-runner` corre restore, build, test y smoke en Podman (`--network=none` , sin capacidades).
6. Si falla la compilación: `MissingUsingFixer` arregla `using` sin modelo (hasta 3 rondas) y el resto vuelve al dueño de cada archivo (hasta 2 rondas).
7. Vera hace la revisión estática y Java calcula `validationStatus` : `VERIFIED` , `FAILED` o `UNVALIDATED` .

Ciclo de producto y lo comercial

Sin producto a la venta, Forjai investiga, construye y después busca clientes a diario; el dinero solo cuenta cuando el fundador lo registra.

Módulo	Qué hace	Quién decide	Pantalla / API
Catálogo (ProductService)	Productos IDEA → IN_CONSTRUCTION → READY_TO_SELL; ProductReadiness exige cliente, precio, evidencia web, validación y margen	Agentes hasta "listo"; fundador pausa, retira, reactiva	/productos
Orquestador (ProductOrchestrator)	Elige idea → completa ficha → lanza misión de construcción; se pausa solo tras 2 ciclos fallidos	Policy ORCHESTRATOR_ENABLED	/productos , /api/company/orchestrator
Prospectos (prospecting)	Sofía busca a diario; Java valida nombre y email en su página; Kira propone estrategias los lunes	Policy PROSPECTING_ENABLED ; estrategias ●	/prospectos
Contacto (outreach)	Borradores validados por Java; envío por SMTP con tope diario y bajas	Cada envío ●	/prospectos
Finanzas (FinanceService)	Clientes, ventas, gastos y correcciones inmutables; ganancias y balance	Solo el fundador registra	/finanzas
Oportunidades	Cada misión crea una Opportunity y guarda candidatos como LEAD	Automático ●	Activity, chat

El modo automático del Dashboard (AutonomyService) prende o apaga juntos el orquestador y la búsqueda de clientes.

Modelos, chat y convenciones

Modelos. CeoService es el único punto que llama a un modelo. Engineering usa kimi-k3 , CEO y Discovery nemotron-3-ultra , Creative y Marketing glm-5.3 , todos vía NVIDIA. Si un modelo remoto cae 2 veces seguidas, ModelHealthService lo marca DOWN y desvía al suplente local (qwen3-coder : 30b) hasta que vuelva. Las keys se editan cifradas desde Settings. Nunca se combinan format y tools en la misma llamada.

Chat. ChatIntentRouter clasifica cada mensaje con reglas antes de tocar un modelo, en este orden:

1. Ejecutar o iniciar una misión.
2. Decisiones del fundador y gobernanza (aprobar correos, pausar productos, modo automático).
3. Menciones @Nombre : responde ese agente, solo lectura.
4. Consultas deterministas (equipos, misiones, profit, status) armadas 100% en Java.
5. Lo demás va a Alex (CEO) con la herramienta query_company_memory .

Configuración. Las variables de entorno solo siembran el primer arranque; después el valor real vive en Neo4j (modelos, policias versionadas, prompts versionados) y se edita en el Command Center.

Convenciones.

- Jackson 3 (`tools.jackson.*`), nunca `com.fasterxml` .
- Sin `@Async` : executors explícitos con `CompletableFuture` .
- Neo4j con driver plano y Cypher a mano; Kafka 100% productor con eventos `EMPRESA_*` .
- Las misiones viven en threads en memoria: no sobreviven un reinicio del contenedor.